CS 4290/6290, ECE 4100/6100 - Spring 2021

Dr. Conte
Project 4 - Cache Coherence

Final Due: 27th **April 2021, 11:59 PM**

Version 1.00

Rules

- **This is an individual assignment. ABSOLUTELY NO COLLABORATION IS PERMITTED**. All cases of cheating will be reported to the Dean of Students.
- The due date at the top of this assignment is final. **It will not be changed**. Late assignments will not be accepted.
- For any questions, please use Dr. Conte's or one of the TA's office hours. If you cannot make it to office hours, please email the TA's.
- This is a tough assignment that requires a good understanding of concepts before you can start writing code. **Make sure to start early**.
- Read the entire document before starting. There may be critical pieces of information and hints along the way.
- Unfortunately, experience has shown that there is a very high chance that there are errors in the project description. **It is your responsibility to check the website often and download new versions of this project description as they become available**
- A makefile with the frontend will be provided. You will only need to fill in the empty functions and any other subroutines you may need.

1. Project Description

On the surface cache coherence seems straightforward; all caches simply must see all operations on a piece of data in thesame order. Implementation of coherence, however, is not so simple. In this project, you will be creating a simulator that maintains coherent caches for a 4,8, and 16 core CMP. You will be implementing the **MSI, MESI, MOSI, MOESI, and MOESIF** protocols for a bus-based broadcast system.

2. Simulator Specifications and More Details

The following list details the simulator specifications, and describes your assignment more information can be found by reading the source code:

- The simulator is capable of simulating a 4, 8, and 16 core CMP system. Each core will consist of a memory trace reader. This trace reader will read in trace files provided to you. The trace reader code will be provided to you.

- Each core in the CMP has one level of cache. The cache is fully associative, has infinite size, and has a single cycle lookup time. The base cache code is provided for you. You will only need to implement the protocol files needed to process requests at the cache (described later). The CMP has a single memory controller which can access the off chip memory. This memory controller is provided for you and will respond to any query (GETS or GETM) placed on the bus with data after a 100 cycle delay.

- The bus modeled is an atomic bus. This means that once a query is placed on the bus, the bus will not allow any other requests onto the bus until a DATA response is seen. Caches request the bus using the bus request() function. If the bus is not available, it will place the request on an arbitration queue to be scheduled in the future. This is done on a first come first served basis, with node 0 having the highest priority and node N having the lowest priority.

- Each processor (trace reader) will have up to one outstanding memory request at a time. The processor will send a request to the cache and will wait until the cache responds with a DATA msg. (This will be done using the send DATA to proc() command)

- We have provided a full simulator framework in C++. This framework creates the simulator, reads in the traces, creates a basic cache structure, and creates the memory controller. All of this code is in the sim/ directory of the downloadable code. You should not need to change ANY code in the sim/ directory.

- The protocols/ directory contains files you may need to modify. Most notably, you need to implement the protocol files. When a request comes from a processor to the cache, the cache finds the entry and then calls process cache request()in the protocol. It is in this function that you should look at the cache entry's state and decide what messages (if any) should be sent, and what state the cache should transition to. When a request is snooped on the bus, the cache finds the entry and then calls process snoop request() in the protocol. It is in this function that you should look at the cache entry's state and decided what messages (if any) should be sent, and what state the cache should transition to.

- To help you in understanding the framework, the MI protocol is already completed and given to you. You MUST fill in the following files: **MSI protocol{.h/.cpp}, MESI protocol{.h/.cpp}, MOSI protocol{.h/.cpp}, MOESI protocol{.h/.cpp}, MOESIF protocol{.h/.cpp}**.

NOTE: In order to interface properly with the Simulator, do not change any of the class names or delete any functions. You may however need to add additional functions, states, and/or messages in order to complete the assignment.

3. Simulator Assumptions

Important Notes About Simulator Assumptions:

1. All requests that are not DATA (GETS and GETM) always expect to have someone reply with DATA. To ensure this, the memory will always respond 100 cycles after the request with DATA unless another cache places DATA on the bus first.

2. There are cases in the traditional protocol where there were certain messages that did not expect replies (e.g. Bus Upgrade). These types of messages are not supported by the bus and memory, so you cannot use them. Instead you should always send a query that expects a data response (e.g. GETS and GETM). This creates situations where the cache sending the GETS or GETM may be the one that should supply the data. In these cases, the cache should simply send DATA to itself on the bus. Given the difficulties in identifying your own GETS or GETM request, the validations assume that you only reply when you are already responsible for supplying the data.

3. In general, there is more than one way to implement each protocol. For this project, the reference implementations were made with simple logic and emphasizing cache to cache transfers, while running in minimal time. For example, contrary to the MESIF article in Wikipedia, the reference implementation will keep the F state at the same processor and not move it.

4. Framework Details

You should download the file labeled **Project4.tar.gz** to get the simulator source code. This framework uses C++. We will be compiling and running your submitted code using a recent version of gcc. You are only responsible for filling in the code in the files in the **protocols/** directory.

Running the Simulator

From the root directory of the project, run the make command and then use the following command to run a trace:

./sim_trace -t trace_directory -p protocol

As an example, you can run:

./sim_trace -t traces/4proc_validation -p MI

Trace directory is the directory with the trace you want to run. A trace directory consists of a trace for each core in the machine and a config file that contains the number of cores for this trace. Each line in the trace directory denotes one memory access and includes the action (read or write) and the address. protocol is the protocol you want to run. The options are: MI, MSI, MESI, MOSI, MOESI, and MOESIF.

Validation: Inside of each trace directory you will find multiple text files for the validation runs of each protocol. You should perform experiments on all of the provided experiment traces.

Statistics

We are concerned about the following statistics:

- Final cache coherence state (Already output by framework)
- Number of cycles to complete execution (Already output by framework) • Number of cache misses (This can be due to a cold miss or coherence)
- Number of cache accesses (Already output by framework)
- Number of silent upgrades for the MESI protocol
- Number of Cache-to-cache transfers (This refers to the number of times data is not supplied by Memory)

5. Validation and Experiments

You must run your simulator and debug it until it completely matches all the statistics in the validation outputs. You will not get credit for your experiments until you do this

Experiments

Compare each of the provided programs (experiment1, ... experiment8) using the various protocols you implemented (MI, MSI, MESI, MOSI, MOESI, MOESIF). Plot the execution time and using the statistics above (and any other information you deem necessary) explain why certain protocols perform better for certain traces.

6. Grading

You will be evaluated on the following criterion:

+0 : You don't turn anything in by the deadline

+50 : You turn in well commented significant code that compiles but does not match the validation

+20 : Your simulator matches the validation outputs of three or more protocols ($3 \leq num < 5$)

+20 : Your simulator matches the validation outputs of all protocols

+10 : Your experiments are completed and submit high quality tech reports. You are unable to get these experimental credits until your simulator matches the validation outputs of all protocols.

7. What to Hand In

You must make a single **tar.gz** file names (your gt account login.tar.gz) (eg. gburdell3.tar.gz) that contains the following:

1. The design results of the experiments for each trace file, with a persuasive argument of the choices that were made in **PDF** format
2. The commented source code for the protocols added to the simulator program.